

# Relatório de Aprendizagem

## Sistema RAG Distribuído via Tailscale — Tema 5

**Aluno:** João Miguel **Disciplina:** Engenharia de Contexto e Harness Engineering aplicada à Programação Distribuída e Paralela **Instituição:** CESUPA — 2º bimestre de 2026 **Período relatado:** 2026-05-23 (sessão de onboarding + Trilha C do Bloco B3)

## Sumário

1. Introdução
2. Contribuição técnica
3. Aprendizados técnicos 3.1 Pipeline de ingestão e estratégia de chunking 3.2 Terraform com provider Docker — IaC sem cloud 3.3 Ansible: bootstrap, deploy e a fronteira com o Terraform
4. Aprendizados de processo
5. Conclusão

## 1. Introdução

Este relatório documenta o que aprendi ao entrar no projeto do Tema 5 da disciplina de Programação Distribuída e Paralela na sessão de onboarding do Bloco B3, como integrante da Trilha C (PC3). O Bloco B1 já estava concluído pelo autor (Davi) e parte do B2 encaminhada, então meu foco foi compreender o sistema como um todo antes de assumir as responsabilidades de infraestrutura como código que me cabiam. Procuro registrar tanto os ganhos técnicos — entender o pipeline de ingestão por dentro, escrever Terraform sem cloud e Ansible com separação clara entre bootstrap e deploy — quanto os de processo, em especial sobre como entrar num projeto a meio sem atropelar decisões já tomadas.

## 2. Contribuição técnica

Fui responsável pela infraestrutura como código da topologia distribuída em três PCs físicos: o módulo Terraform em `infra/terraform/` com provider `kreuzwerker/docker` (root module, módulos `server` e `worker`, variáveis tipadas em `variables.tf` e `.tfvars` por host); o `playbook-bootstrap.yml` que instala Docker, Tailscale e — só no PC1 — o NVIDIA Container Toolkit; o `playbook-deploy.yml` que sincroniza o repositório via `rsync`, gera o `.env.local` a partir de um template Jinja2 e executa `terraform apply` em cada host; as roles `docker`, `tailscale` e `nvidia`; o inventário `infra/ansible/inventory.yml` como fonte de verdade da topologia; e o wrapper `scripts/deploy.sh` que encadeia os dois playbooks. Antes de escrever uma linha de IaC, percorri o `CLAUDE.md`, o `TODO.md`, os planos B1-B5 e o código-fonte dos workers de ingestão para garantir que o código de infraestrutura refletisse a arquitetura real do projeto, e não uma interpretação minha dela.

## 3. Aprendizados técnicos

### 3.1 Pipeline de ingestão e estratégia de chunking

- **Por que chunking existe:** associava “dividir texto” a um `split(" ")` antes deste projeto. Aqui aprendi dois motivos reais — granularidade do retrieval (uma página inteira como vetor único diz “este documento é relevante” mas não “este parágrafo responde X”) e qualidade do embedding (vetores de 768 dimensões viram “média” do conteúdo em textos longos, perdendo especificidade).
- **Overlap como redundância barata:** 120 tokens repetidos entre chunks adjacentes parecem desperdício até perceber que uma frase importante na fronteira seria cortada sem isso. O overlap paga em recall — nenhum dos dois chunks vizinhos fica órfão da frase.

- **Idempotência por construção:** o id de cada ponto no Qdrant é derivado de sha256(doc\_id:chunk\_index) então reenviar o mesmo documento sobrescreve os mesmos pontos em vez de duplicar o corpus. É uma decisão de design que dispensa coordenação explícita entre tentativas.
- **Profile vs nomeação explícita no Compose:** nomear um serviço diretamente em docker compose up anula a restrição de profile. Permite subir só rabbitmq qdrant ollama ingest-worker sem o sistema todo, mantendo depends\_on funcionando normalmente.

### 3.2 Terraform com provider Docker — IaC sem cloud

- **Declarativo vs imperativo:** minha experiência anterior com Terraform era superficial e sempre associada à AWS. O conceito que ficou claro aqui é que o Terraform descreve o estado desejado, não os passos para chegar lá — declaro “quero um container rag-rabbitmq com estas variáveis e portas” e ele calcula o que precisa criar, modificar ou destruir.
- **Topologia em módulos condicionais:** modules/server descreve o PC1 (RabbitMQ, Qdrant, Redis, Ollama, gateway); modules/worker descreve PC2/PC3 (ingest-doc, ingest-chunk, query-worker apontando para o PC1 via IPs Tailscale). O main.tf instancia ambos com count = var.host\_role == "..." ? 1 : 0, então o mesmo código serve nos três PCs — só o .tfvars muda.
- **plan é seguro, apply é destrutivo:** terraform plan mostra o que *seria* feito sem fazer nada. É o comando que vale mostrar no doc técnico como prova de que o IaC está correto — não preciso aplicar de verdade para demonstrar.
- **Não compartilhar daemon com o Compose:** Terraform e Docker Compose não devem gerenciar o mesmo daemon ao mesmo tempo porque os dois lêem e escrevem no Docker como fonte de verdade. Em dev local (Modo 1) usa-se o Compose; em distribuído (Modo 2), o Terraform. Os dois caminhos existem, mas nunca ao mesmo tempo no mesmo host.

### 3.3 Ansible: bootstrap, deploy e a fronteira com o Terraform

- **Bootstrap vs deploy como dois playbooks:** playbook-bootstrap.yml prepara o **host** (Docker, Tailscale, NVIDIA Container Toolkit no PC1) — operação de “primeiro dia”, become: true. playbook-deploy.yml implanta a **aplicação** (rsync, render do .env.local, terraform apply) — operação repetível, become: false. Misturar os dois tornaria cada reimplantação desnecessariamente lenta.
- **Roles como reutilização:** a role docker é definida uma vez e referenciada onde precisar; a role nvidia existe mas só é chamada no play que aponta para hosts: pc1. Os outros hosts nunca a executam, sem código duplicado.
- **Templates Jinja2 para configuração por host:** em vez de copiar um .env.local fixo, o Ansible renderiza env.j2 em tempo de deploy com os valores do inventário. pc1.ansible\_host (o IP Tailscale real do PC1) entra automaticamente nas URLs de todos os workers — sem edição manual em cada máquina.
- **Por que Terraform e Ansible:** Ansible é orientado a tasks (instalar pacotes, gerir utilizadores, copiar ficheiros); Terraform é orientado a estado declarativo (containers, redes, volumes). Aqui o Ansible prepara o terreno e dispara o Terraform em cada host; juntos formam a cadeia host pronto → containers declarados → aplicação rodando.

## 4. Aprendizados de processo

- **Entrar a meio é diferente de começar do zero:** o projeto tem documentação densa (CLAUDE.md, TODO.md, spec, planos de bloco) e ler isso antes de tocar no código poupou tempo. O TODO.md foi o mais útil — mostra não só o que está feito, mas quem fez (trilha), quando (bloco) e qual foi o marco de validação. Sem esse mapa, tentar entender a arquitetura só pelo código seria muito mais lento.
- **Trilhas e blocos são ortogonais:** blocos são o **quando** (fases cronológicas — B1 setup, B2 pipeline, B3 tolerância + IaC); trilhas são o **quem** (A autor/PC1, B colega1/PC2, C eu/PC3). Perceber essa separação cedo evitou colisão com o trabalho dos colegas e tornou claro o que me competia em cada bloco.

- **laC é documentação executável da topologia:** antes deste projeto, via laC como “automação para não clicar na consola”. Depois de escrever o Terraform e o Ansible, a utilidade maior virou outra — o código descreve a topologia do sistema de forma versionada e verificável com `terraform plan`. Quem ler `modules/worker/main.tf` sabe o que roda em PC2/PC3 sem precisar perguntar.
- **Cortes pré-definidos como decisão de design:** o spec define uma lista de cortes em ordem prioritária caso o cronograma aperte. O Terraform está no último lugar, ou seja, é o primeiro a cair. Entender isso antes de começar mudou minha abordagem — fiz o suficiente para o entregável acadêmico (estrutura correta, `terraform plan` funciona) sem over-engineering. Definir o que *não* se vai fazer também é parte do design, não desistência.

## 5. Conclusão

Entrei no projeto num momento crítico — dois dias antes da entrega, com a infraestrutura distribuída por fazer. A sessão de onboarding forçou-me a entender o sistema antes de escrever uma linha, e isso custou tempo no início mas poupou muito mais depois: quando comecei o Terraform, já sabia exatamente o que cada container precisava porque tinha percorrido o `docker-compose.yml` e o código dos workers. O aprendizado mais duradouro não é a sintaxe do Terraform nem os módulos do Ansible — é a ideia de que **laC é documentação da topologia, não só automação**. O código que escrevi é a resposta executável à pergunta “como é que este sistema distribuído está organizado nos três PCs?”, e qualquer colega que clonar o repositório lê o `modules/server/main.tf` e sabe o que roda no PC1 sem ter que perguntar. Os próximos dias serão a parte mais densa — rodar os experimentos, interpretar resultados e escrever o documento técnico — mas é onde a infraestrutura que configurei serve de palco para o que o sistema realmente faz.